CodeMafia 🕵️‍♂️💻

1. Project Overview & Pitch

The Elevator Pitch:

Among Us meets LeetCode. CodeMafia is a real-time, browser-based social deduction game for 3-5 players where debugging is the actual gameplay.

The Concept:

Players join a multiplayer lobby and are presented with a broken Python algorithm and a set of failing test cases. The team must collaborate in real-time to fix the code and pass the tests. However, there is a twist: one player is secretly the **Impostor**. The Impostor's goal is to subtly sabotage the logic, introduce infinite loops, or break edge cases without getting caught.

Civilians win by making all tests pass or voting out the Impostor. The Impostor wins by completing their sabotage objectives or surviving until the timer runs out.

Why This Project Stands Out:

- **Highly Interactive:** Goes beyond standard CRUD apps by introducing real-time web socket communication.
- **Technically Impressive:** Utilizes WebAssembly (Wasm) to securely run Python code directly in the browser, eliminating the need for expensive backend sandboxing.
- **Strong Algorithmic Focus:** Perfect for showcasing Data Structures and Algorithms (DSA) skills in a fun, gamified environment.

2. Tech Stack

This stack is chosen to maximize learning in modern, high-demand web technologies while keeping the infrastructure manageable.

Frontend (The Client)

- **Framework:** React (bootstrapped with Vite for maximum speed).
- **Language:** TypeScript (critical for managing complex game states and ensuring player objects are strictly typed).
- **State Management:** Zustand (lightweight global state for the active game phase and player lists).
- **Styling:** Tailwind CSS (for rapid UI development) & Framer Motion (for smooth, game-like animations).

Core Mechanics (The Editor & Execution)

- **Code Editor:** `@monaco-editor/react` (Provides a genuine VS Code experience in the browser, including syntax highlighting).
- **Code Execution:** Pyodide (Compiles Python to WebAssembly, allowing code and test cases to run instantly and securely inside the user's browser).

Backend (The Multiplayer Server)

- **Environment:** Node.js with Express.
- **Real-Time Engine:** Socket.io (Handles bidirectional event-driven communication, player rooms/lobbies, and broadcasting game state).

3. Building Roadmap

To avoid getting stuck, this project must be built in a specific sequence. **Do not build the UI or backend first.** Always start with the core mechanic.

Phase 1: The Core Engine (Single-Player Proof of Concept)

Goal: Prove you can edit and run Python in the browser.

- Set up a basic React + Vite + TypeScript project.
- Integrate the Monaco Editor component.
- Integrate Pyodide.
- **Milestone:** Write a simple Python function in Monaco, click a "Run" button, and have Pyodide evaluate it against a hardcoded test case (e.g., `assert add(2, 2) == 4`) and display "Pass" or "Fail".

Phase 2: The Networking Foundation

Goal: Connect multiple browsers together.

- Initialize a Node.js + Socket.io server.
- Create a simple React UI to connect to the Socket server.
- Implement standard Socket.io "Rooms" logic (Create Room, Join Room with a code).
- **Milestone:** Open two browser windows. When Window A joins Room "1234", Window B (also in Room "1234") sees the player list update in real-time.

Phase 3: Collaborative Editing (The Sync)

Goal: Share the code state across sockets.

- Listen for `onChange` events in the Monaco Editor.
- Emit these text changes to the Socket.io server.
- Broadcast the updated text to all other clients in the room.
- *Note: For a basic implementation, simple state broadcasting works. If you encounter cursor jumping, look into basic Operational Transformation (OT) or debounce the inputs.*
- **Milestone:** Two players can type in the same editor and see each other's changes.

Phase 4: The Game Loop & Roles

Goal: Turn the sandbox into a game.

- Define TypeScript interfaces for your Game State (Lobby -> Playing -> Meeting -> End).
- Write backend logic to randomly assign one Impostor when the game starts.

- Lock the editor for the Impostor (or give them a separate secret task list), while Civilians see the main TODOs.
- Implement win/loss conditions (Timer runs out = Impostor wins; Tests pass = Civilians win).
- **Milestone:** A full game can be played from lobby creation to a win/loss screen.

Phase 5: Social Deduction (Meetings & Voting)

Goal: Add the "Among Us" mechanics.

- Add an "Emergency Meeting" button that locks the code editor for everyone.
- Create a simple chat interface that only works during meetings.
- Implement a voting system. Send votes to the backend, tally them, and broadcast who is eliminated.
- **Milestone:** Players can pause the game, argue, and eliminate a player.

Phase 6: Polish & Deployment

Goal: Make it look and feel like a finished product.

- Add Framer Motion for dramatic pop-ups (e.g., "EMERGENCY MEETING", "YOU ARE THE IMPOSTOR").
- Add simple sound effects for test passes, chat messages, and game over.
- **Deployment:** Host the frontend on Vercel or Netlify, and the Node.js backend on Render or Railway.